

4. Simple Note App Creation - Detailed Guide

4.1 Overview

Features

- Create notes with title and content
- View all notes in scrollable list
- Edit existing notes
- Delete notes with confirmation
- Persistent local storage
- Timestamp tracking

Tech Stack:

Sqflite

SQLite for local database

Provider

State management solution

path_provider

Get device storage paths

Step 1: Project Setup

Folder Structure

```
.
├── lib/
│   ├── models/
│   │   └── note.dart
│   ├── database/
│   │   └── db_helper.dart
│   ├── providers/
│   │   └── note_provider.dart
│   ├── screens/
│   │   ├── home_page.dart
│   │   └── note_editor_screen.dart
│   ├── widgets/
│   │   └── note_card.dart
│   └── main.dart
```

pubspec.yaml

```
dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.8
  sqflite: ^2.3.0
  path_provider: ^2.1.0
  provider: ^6.1.0
  intl: ^0.18.0
```

Step 2: Note Model

Model Features

- JSON serialization methods
- Database mapping functions
- Nullable ID for new notes
- Timestamp handling

models/note.dart

```
1 class Note {
2   final int? id;
3   final String title;
4   final String content;
5   final DateTime createdAt;
6   final DateTime updatedAt;
7
8   const Note({
9     this.id,
10    required this.title,
11    required this.content,
12    required this.createdAt,
13    required this.updatedAt,
14  });
15
16   Map<String, dynamic> toMap() => {
17     'title': title,
18     'content': content,
19     'createdAt': createdAt.toIso8601String(),
20   };
21
22   static fromMap(Map<String, dynamic> map) {
23     return Note(
24       id: map['id'],
25       title: map['title'],
26       content: map['content'],
27       createdAt: DateTime.parse(map['createdAt']),
28       updatedAt: DateTime.parse(map['updatedAt']),
29     );
30   }
}
```

Step 3: Database Helper

Singleton Pattern

Use singleton to ensure only one database instance exists throughout the app lifecycle.

```
1  import 'package:sqflite/sqflite.dart';
2  import 'package:path/path.dart';
3
4  class DatabaseHelper {
5      static final DatabaseHelper instance = DatabaseHelper._init();
6      static Database? _database;
7
8      DatabaseHelper._init();
9
10     Future<Database> get database async {
11         if (_database != null) return _database!;
12         _database = await _initDB();
13         return _database!;
14     }
```

Step 4: CRUD Methods

```
41 // Create
42 Future<int> create(Note note) async {
43     final db = await database;
44     return await db.insert('notes', note.toMap());
45 }
46
47 // Read All
48 Future<List<Note>> readAll() async {
49     final db = await database;
50     final maps = await db.query('notes', orderBy: 'updatedAt DESC');
51     return maps.map((m) => Note.fromMap(m)).toList();
52 }
53
54 // Update
55 Future<int> update(Note note) async {
56     final db = await database;
57     return await db.update(
58         'notes',
59         note.toMap(),
60         where: 'id = ?',
61         whereArgs: [note.id],
62     );
63 }
64
65 // Delete
66 Future<int> delete(int id) async {
67     final db = await database;
68     return await db.delete('notes', where: 'id = ?', whereArgs: [id]);
69 }
70 }
```

Step 5: State Management

Provider Benefits

- Centralized state
- Automatic UI updates
- Easy data sharing
- Testable logic

```
1  import 'package:flutter/material.dart';
2
3  import 'database_helper.dart';
4  import 'note_models.dart';
5
6  class NoteProvider extends ChangeNotifier {
7    List<Note> _notes = [];
8    List<Note> get notes => _notes;
9
10   Future<void> loadNotes() async {
11     _notes = await DatabaseHelper.instance.readAll();
12     notifyListeners();
13   }
14
15   Future<void> addNote(Note note) async {
16     await DatabaseHelper.instance.create(note);
17     await loadNotes();
18   }
19 }
20
```

Step 6: Main App

ChangeNotifierProvider

Wraps the entire app to make state available everywhere in the widget tree.

```
1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(const MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8   const MyApp({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return MaterialApp(
13      title: 'Flutter Demo',
14      theme: ThemeData(
15        colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),
16      ),
17      home: const HomePage(),
18    );
19  }
20 }
```


Step 7: Home Screen

AppBar with title

ListView for notes

FloatingActionButton to add

```
33  class _HomePageState extends State<HomePage> {
34      @override
35      Widget build(BuildContext context) {
36          return Scaffold(
37              body: SizedBox.expand(
38                  child: Consumer<NoteProvider>(
39                      builder: (context, provider, child) {
40                          return ListView.builder(
41                              itemCount: provider.notes.length,
42                              itemBuilder: (context, index) {
43                                  final note = provider.notes[index];
44                                  return NoteCard(note: note, onTap: () => _editNote(note));
45                              },
46                          );
47                      ),
48                  ),
49              ),
50          );
51      }
52
53      void _editNote(Note note) {
54          // TODO
55      }
56  }
```

Step 7: Home Screen

Now Your Turn

Please complete this app and submit your repository on TDMU E-Learning to get extra points.